Managed capability declarations

This page owns the `features` declaration and its effective defaults. Individual capability pages own their interfaces and application-level prerequisites.

Schema and defaults

`PUT <config-plane>/config/features` takes `{"features": {...}}`. The only accepted keys are optional booleans `server` and `database`; unknown keys are rejected.

For **Web projects**, both templates (`flexible` and `web-db-user`) use the same resource selection in Cloud and Local. New projects start with both capabilities off:

Declaration	Effective server	Effective managed database
Omitted `features` or `{}`	Off	Off
`{"server": true}`	On	Off
`{"server": true, "database": false}`	On	Off
`{"database": true}`	On (required dependency)	On
`{"server": true, "database": true}`	On	On

On later Web config writes, omitted feature keys preserve their existing effective values; they neither request a new database nor remove an existing capability. This applies to both whole-document and features-domain writes. Requesting `database: true` while omitting server
also enables server if needed. A successful config response includes a `note` when that call automatically enabled server; initialization reports the dependency in its tool result. Explicit `database: true, server: false` is invalid (`needs_server_true`).

Game and Mobile retain their existing policies. Game initializes with both off and supports later upgrades through config; its features-domain writes replace the declaration and `server: true` with omitted database still enables both. Declare both values explicitly for Game upgrades. Mobile initializes with both on and requires both to remain on.

All managed capabilities are **one-way enablement**. An explicit request to disable an enabled
capability is rejected as `managed_feature_disable_unsupported` or an `invalid_value` field error on `features.*`. Stored historical declarations retain their effective meaning; editing a Web project does not delete its existing managed resources.

What enablement changes

- `server` selects a server-capable published project. It does not choose a language, create application code, or start the development HTTP process. A static publish uses `build` without a server; a server project uses `deploy`, optionally with `build` for hybrid output. Those artifact contracts belong to static publishing and container publishing.
- `database` provisions the managed MySQL capability. It does not create application tables, migrations, or a migration tool. The database's connection, data scope, and interaction with a user-supplied database are in database.
- Accepted enablement provisions the capability and supplies its platform values. `feature_reconcile_in_progress` identifies provisioning already in flight. A stored declaration, provisioning completion, and a running process's environment are distinct states.

The bound MCP transport reports development environment loading through `runtime_sync`; configuration owns that receipt and existing-process behavior.

Manus application login is not a `features` key; `features.userAuth` is retired. Authentication owns its actual prerequisites. Stripe is an integration, not a feature flag; payments owns its enablement contract. A feature write does not itself require reading either capability's usage guide when the task does not use it.

Declaration procedure

Read the relevant capability contract before declaring it; do not guess unsupported feature keys or copy a remembered legacy schema. Warn the user that managed enablement is one-way before asking the owner to enable it. Keep an enabled feature on after a refused disable; if the requested product needs a separate configuration that cannot be represented by this project, explain that boundary rather than looping on the same disable request.

For Web projects requiring Manus application login, declare `server: true` when creating the project; do not defer discovering that requirement until OAuth implementation. Games initialize with server and database off and enable required services through config only after the online choices are approved in Blueprint. Read authentication before writing that integration. A per-domain `deploy` write requires `server: true` already stored; a whole-document update can contain both atomically. Do not try repeated deploy writes before satisfying that prerequisite.

Before any managed database write, read database: development and published application share that database. Declaring `database: false` is for a project that has not already enabled it, not a way to dismantle a live managed database.

Prepare a static build declaration as soon as the site builds, and a container deploy declaration with its committed Dockerfile when the server listens. Do not wait for a publish request.
The detailed build reference covers runtime-specific needs and build failures.

Read payments before enabling Stripe **or writing its application integration**.
For another provider, identify the provider's contract and required protected values first; do not invent a platform feature flag. Secret declarations and revisions require reading secrets; protected value collection still uses its registered user input surface.